FILE SYSTEMS

One of the main purposes of most computer systems is to process information or data and to make the result of this processing known or available to the user. This implies that the user must be able to enter data, as well as save or retrieve the result. This will be done by means of files. For the majority of modal computer users, a file is the most visible concept of the operating system. In this chapter, we will discuss some aspects of files that most users (and programmers) will come in contact with frequently, and we will also take a look at the underlying techniques that are used to implement a file system so that it achieves good performance. Since files are usually stored on disks, it is useful to keep in mind the working of the disk scheduling algorithms discussed earlier.

1 File attributes, operations and types

Information on a computer system will always be organized in files, which are usually stored on a disk (or flash memory). All user data, no matter how small, will consist of one or more files. The content of a file is typically a collection of bits, bytes, lines or records. Depending on the content of the file, one speaks of *text*, *source*, *object* and *executable* files.

Attributes

Although the contents of a file may vary greatly, all files have some similar attributes:

- **File name:** For the convenience of the user, each file will have a name, so that they are easily identifiable and readable. In older systems, the name of a file often had to meet fairly strict requirements.

Nowadays there is still a maximum file name length. However, this is so large that the length of the file name seems unlimited for practical purposes.

- **Type:** File types distinguished by the operating system are unrelated to the known *file extensions* (e.g. .doc, .txt). File systems such as Unix and Linux use a very limited number of file types (see below).
- **Location:** This information identifies the I/O device on which the file resides and further indicates the location of the file on the device.
- **Size: The** size of the file is also recorded (in bytes or blocks). For most systems, there is also a maximum file size, which can be significantly smaller than the disk capacity.
- **Protection:** Since the data of multiple users often has to share a single disk, there is also a need for protection. This info indicates who is allowed to read, write, execute, etc.
- **Time, date and user id:** Usually, information about the owner of the file, the time the file was created and when it was last modified, etc. is also kept.

All of this information for each of the files will be stored in the *directory structure* of the file system, which is also stored on disk. We will return to each of these attributes, as well as the possible organizations of the directory structure, when discussing the various components of a file system.

Operations

First, we briefly mention the different operations that can be performed on a file. Although everyone is undoubtedly familiar with this matter, we will mention them here to point out the underlying file system actions that the operating system must support.

First of all, one must be able to **create** a file. For this, free space on the disk must be found. Hence, there is a need for *free-space* management, which will

inventory and allows you to quickly locate free disk space. We will come back to this later.

Furthermore we need to be able to **read** or **write** information from a file. To realize this the file must first be localized. This is done by searching the directory structure. Furthermore, often a read or write pointer will be kept which indicates at which position in the file the data must be read or written. This position is in this case *no* input parameter of the read or write operation. After each operation, this pointer is adjusted. Since a process usually does not read and write a file simultaneously, the read and write pointer will usually be combined in the *current-file-position* pointer.

Another useful operation is the **file seek**, which allows to change the value of the current-file-position. Of course, this operation also requires that the file be located first. Files can also be **deleted**. After localizing the file, the necessary information is removed from the file directory and the space that the file still occupied on the disk must be taken back by the free-space management mechanism. Other operations like adding data to the end of a file or renaming a file are sometimes also part of the basic operations of a file system. The other operations, like copy, are usually offered as a combination of the previous ones.

Open-file table(s)

What is striking about the previous discussion is that most, if not all, operations require us to search the directory structure. To avoid having to do this during each operation we use an *open-file table*. Before we perform an operation on an existing file, this file will be opened first (this can be done implicitly or explicitly). The open-file table keeps information about all open files.

When opening the file, the directory entry of the file is first located. This is then (if the protection of the file does not prevent this) copied to the open-file table. The index of the open-file table indicating where this entry is located will be returned as a return argument to the open operation (in case opening a file is an explicit operation). All I/O operations will then use this index and will no longer use the file name. This avoids repeatedly searching the directory structure.

Usually there is also a close operation, although the file is usually also automatically removed from the open-file table if the process that opened it ends.

The open and close operations require a bit more work if we work in a multiuser environment, because sometimes several processes can open, read, etc. the same file. Therefore we work with two levels of open-file tables. At the highest level, there are *per-process* open-file tables. For each process such a table is kept. It indicates which files are currently in use by the process. To support the simultaneous reading of a file by different processes, a current-file-position pointer is kept for each process. The value of this pointer is also stored in the per-process table.

A pointer to an entry in the *system-wide* open-file table is also given. provided. All information about the open file that is identical for the processes that are currently using the file can be found there (such as the file location, size, dates, protection information, etc.). Additionally, an *open-file counter* is kept. This counter indicates how many processes are currently using the file. With every open (close) operation this counter is increased by
increment (decrement). If the counter reaches zero, the entry is removed from the system-wide open-file table.

I/O memory mapping

The information regarding the location of the opened files is usually stored in memory, so that fast access can be supported. Furthermore, files are also mapped in the virtual memory of a process (see chapter Virtual Memory). We speak of *memory mapped files*. This way, information can flow very fast from and to the file without the need for disk access (if the corresponding pages are present in memory). Reading and writing operations to the file, are similar to reading and writing a word in virtual memory. Only when the file is closed, the data is written to disk and the allocated virtual memory is released again.

If the operating system uses page tables to support virtual memory, it is very easy to share a file this way. Namely, the virtual addresses that correspond to the file will be translated, through the page table, to the same page frame, which is (a

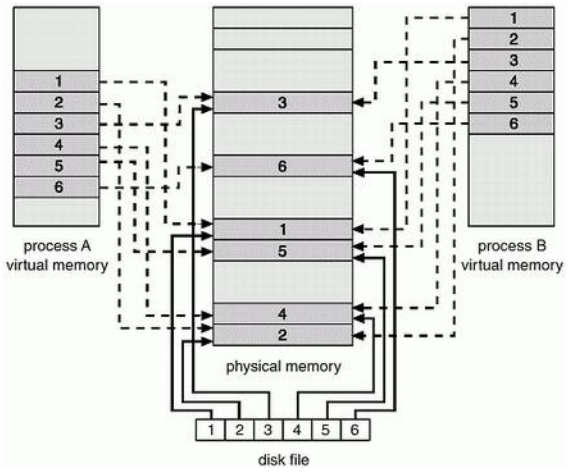


Fig. 54: Memory mapped files [Silberschatz 94].

part of) the file (see Figure 54). This is important because different processes will very often use the same dynamic libraries (that is, .dll or .so files).

File types

When we talk about the type of a file, we immediately think of the file extension (e.g. exe, .com, .bat, .obj, .o, .c, .f77, .txt, .doc, .tex, .ps, .dvi, .gif, .zip, .tar, etc.). These extensions are for convenience of the user and the applications associated with them, and are independent of the file types recognized by the file system. When a file system uses multiple file types, the internal structure of each file type will also be different.

In Unix or Linux, all *regular* files are simply a collection of 8-bit bytes, without any associated interpretation. In short, we work with a single file type. There are some additional file types to support special files, such as directory files, device files, symbolic links, pipes, sockets, etc. Device files are used to write data to or from an I/O device and allow us to write data to a file or an I/O device in the same way. Pipes and socket files allow us to exchange data between processes (whether or not on the same machine). Symbolic links will be discussed later. The type of the file is determined by the first character in the output of the **ls** -l command.

2 The directory structure

Given the large capacity of today's disks, one will often divide the capacity of a disk into a limited number of *partitions* or *volumes*. Each volume has its own directory structure and can therefore be considered a virtual disk. As indicated earlier, the directory structure contains an entry for each file that (indirectly) stores the values of all file attributes. The directory structure must also allow for the quick location of the entry corresponding to a file name (we will return to this topic later).

2.1 Tree structured directories

To organize the files of different users and applications a *tree structure is* often used (see Figure 55). One implements such a structure by associating a directory file with each node in the tree. The first file, which corresponds to the root of the tree, contains, among other things, a set of pointers that each point to one of the directory files corresponding to the level 1 nodes of the tree. These level 1 nodes are the subdirectories of the root node. Analogously, each subdirectory can in turn contain its own list of subdirectories. Furthermore, a directory file naturally contains all the information about the actual data files located in this directory.

When a user creates a new file, its name must be unique only within the directory where it is created. Consequently, a file is uniquely identified by specifying the *path* to take within the tree to locate the file. For example, the file *exp* at level 2 of the tree has as path */spell/mail/exp*. If we need information about this file, we first go through the directory file / to locate the entry that corresponds to *spell*. This contains a pointer to the directory file */spell*, where we look for the entry *mail*. In the directory file where the entry *mail* points to, we finally find the entry *exp* which contains the actual file info. When referring to a certain file from the *shell*, only the directory file of the current directory will be searched. This gives a much faster search result, but also has some limitations.

Suppose several users want to use the same application (e.g. a file editor). In this case, we want to avoid that the necessary

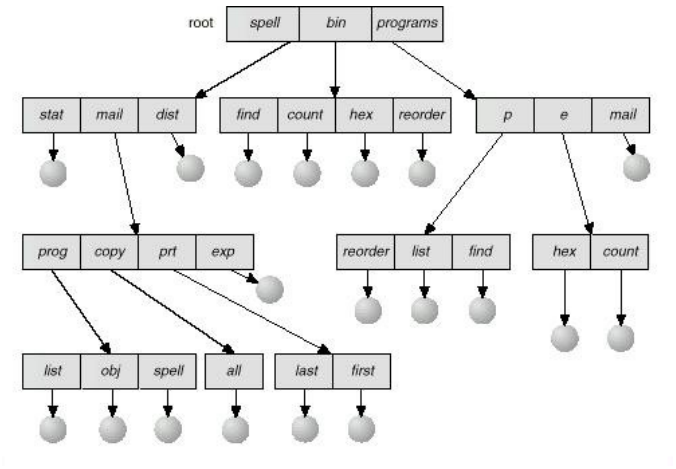


Fig. 55: Tree-structured directory [Silberschatz 94].

file(s) must be present on the system several times. On the other hand, we don't want users to have to provide the full path name every time. To solve this problem the shell uses a *search path*. The search path contains a number of directory names. When a user issues a command from the shell, it will search the current directory as well as the search path. This way all users can just use the short file name, by putting the directory that contains the application in the search path. Since the search path means users don't have to remember long path names, this allows us to easily relocate the files. Besides specifying the full path, you can also work with relative paths (relative to the current directory).

2.2 Excavated structured directories

A tree structure is simple, but it is not always sufficient. Suppose all (or some) users are allowed to read data in a directory belonging to another user, called user A, and do so frequently. Then it would be convenient for them to have quick access to this directory (or files) without having to work with path names or search paths. One solution would be to offer the directory of user A as a subdirectory to all other users.

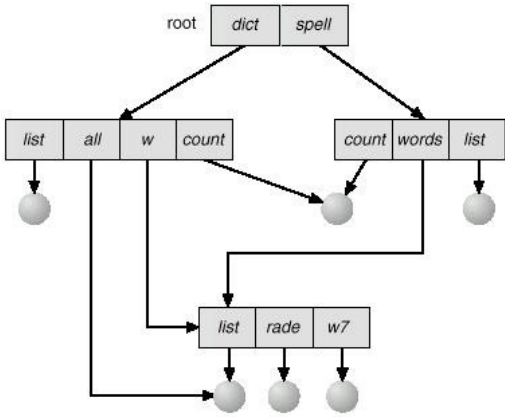


Fig. 56: Excavation-structured directory [Silberschatz 94].

These kinds of subdirectories can be supported by *symbolic soft links*. An entry in a directory file that corresponds to a symbolic link looks similar to an entry for any other directory, except that it also contains a path name indicating the directory to which the link points (the same is true for symbolic links to files). In UNIX, you can create symbolic links using the command **ln** -s.

One of the consequences of providing symbolic links is that a file no longer has a unique path name. This also indicates that the directory structure is no longer a tree, but a directed graph (see Figure 56). This graph can contain closed paths, unless the file system prevents the creation of a closed path. This is important because one often wishes (e.g. for backup purposes) to go through part of the directory structure.

If closed paths can occur, the procedure that runs the structure must be able to deal with them (e.g. in UNIX the command **du** will show the structure of the directory, the cur- rent directory being the root, for symbolic links the -L option is important). In many cases, there will simply be a limit to the number of (symbolic) links one can take in a single path. If a locked path is present, this maximum is exceeded and a warning is issued. By default, procedures that run through directories will not follow symbolic links, so that closed paths cannot be created. The use of symbolic links also raises a number of questions. In-

if we follow a symbolic link and then we go to the parent directory, where do we end up (this seems to be shell dependent)? Deleting files also raises some issues. Can we just physically delete a file (or directory) when references to it still exist? If we want to be sure that there are no more references to the file, we should at least keep a counter that indicates how many links to the file are currently present. When creating the file, this counter is initialized to one. If a delete lowers the counter to zero, we can also physically delete the file (freeing up disk space). In UNIX, *hard* links will take the latter approach, while *soft* links will not prevent the file system from physically deleting a file or directory. In the case of a hard link, the file entry will be taken from the directory structure. Finally, one can usually only establish hard links to a file.

2.3 Directory structure implementation

The main task of the directory structure is to locate the file entry corresponding to a file name. This search can take place in various ways depending on how the entries in the directory files are organized.

A first, simple possibility is to keep the directory as a linear list. This is easy to implement, but it requires that every search has to search the (entire) file until the requested file name is found. When creating a file, the entire list must be checked, because the newly selected file name must be unique within the directory. There are several ways to delete a file. First, you can mark the entry as unused. New files can then take these entries back if necessary. You can also replace the deleted entry with the last entry in the directory structure, this way the directory will become smaller after deletion.

A second important way of managing a directory is to use a more complex data structure so that searching and tries can be done in logarithmic time. Hence, many file systems use balanced trees for this purpose.

A third possibility is to use hash tables. The biggest difficulty here is to make a good choice for the number of possible

results for the hash function (corresponding to the size of the hash table). Large directories need a larger table, while small directories would require too much space if we were to greatly overestimate the size of the table. An intermediate possibility is that each result refers to a linear list of files.

3 Access and allocation methods

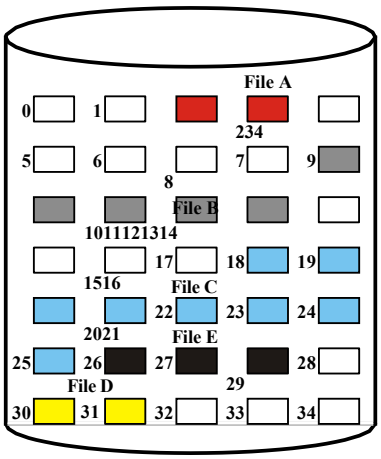
Reading from and writing to a file can be supported in a number of ways. A first, very common, possibility is **se- quential** access. This indicates that the contents of a file are read in order from the first logical record to the last. The current-file-position pointer indicates the current position in the file. Information is also written to the end of the file. This method seems inflexible, but is very suitable for applications like editors, compilers, images, etc. Some file systems also allow sequential ac- cess to jump *n* records forward or backward.

For applications that manipulate a lot of data and have sporadic

If you have to perform read and write operations, sequential access is clearly very inefficient (think of databases). In this case, one will use **direct** or **relative** access. To identify the required data, one gives the number of the logical record one wants to read or overwrite (as input parameter). Direct access is usually combined with a search method allowing to quickly determine in which logical record(s) the data to be read or written is located. Index files can be used in combination with a binary search algorithm or a hash function. When the index file is small enough, it can always be present in memory.

3.1 Allocation methods

Suppose a file has a total size of *n* blocks (or physical records). Then the allocation method will indicate how these *n* blocks will be placed on the disk. There are three main allocation methods that we will discuss: *conti- nuous*, *linked*, and *indexed* allocation. Later in this chapter, we will also look at the allocation technique used by UNIX systems, which is actually a variation of indexed allocation.



Directory

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Fig. 57: Continuous file allocation [Stallings 98].

Continuous allocation

In case of continuous allocation the n blocks of a file are all sequentially located on the disk. If we number the disk blocks starting from zero, then the file will occupy blocks $b, b + 1, \ldots, b + n - 1$ for a given b, which represents the starting address of the file on disk. In this case, the file's directory entry contains the address b as well as the number of blocks the file requires (see Figure 57).

Clearly, this allocation method provides very good support for both sequential and direct access. Since sequential blocks are located next to each other on the disk, almost no seek time is required once the first block is found. For direct access, the block number can be quickly found by adding the required relative block number to b, the starting address.

Continuous file allocation confronts us with similar problems as dynamic partitioning of memory. How do we determine where to place the file? Again, we have a number of options like Best-fit, First-fit, Next-fit or even Worst-fit for selecting the empty space. Another problem is that we need to know the size of the file beforehand. This is often very problematic because a file can grow over a long period of time or because it is an output file for which we do not immediately know how much output will be generated. However, without knowing the size it is difficult to make a choice.

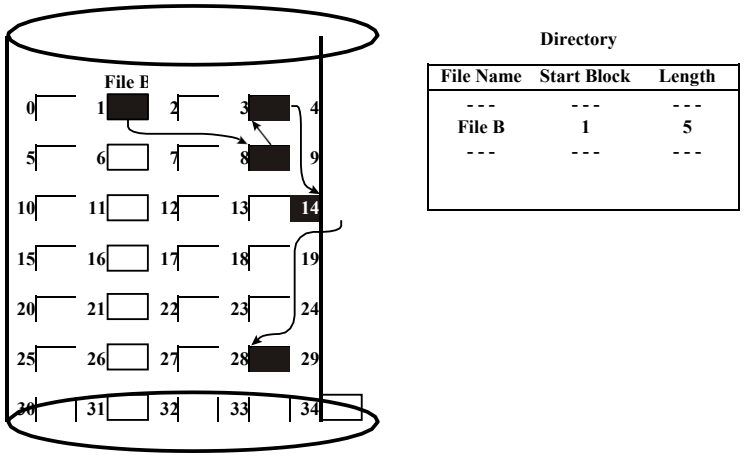


Fig. 58: Linked (or chained) file allocation [Stallings 98].

when choosing a void. Unless we want to support frequent moving of the file, the user should already specify a size when creating the file. This will usually result in an overestimation, which also causes internal fragmentation.

Sometimes, the file system will also offer a routine that allows you to remove the empties from the disk (for example, this was often the case with old floppy disks that used continuous allocation). Of course, running this routine many times is very taxing on the system.

Finally, some systems will allow one or more *extents to* be added to a file. In this case a second, continuous set of disk blocks will be allocated if the original set is no longer sufficient. The disadvantage here is that usually the user has to determine the size of the extent.

Linked (or chained) allocation

With linked allocation, we can get rid of all the problems of continuous allocation. The idea is to store the *n* blocks that make up the file as a linked list. This way the *n* blocks can be spread all over the disk. The directory entry contains a pointer to the first block, this block contains a pointer to block number 2, etc (see Figure 58).

When creating a new file, an entry is first made in the

directory, then for each block the *free-space* algorithm is called to generate a new free block. Each block contains a pointer to the next one. There is no need to know the size of the file in advance. Consequently, the external fragmentation typical of a continuous allocation is no longer applicable. The main disadvantage of this method is that both sequential and direct access give rise to relatively large seek times, especially when the different blocks, produced by the free-space algorithm, are far apart. Furthermore, in each block a limited number of bits must be sacrificed to support the pointer to the next block. (e.g. 4 bytes per 512 bytes).

One can partly avoid the disadvantages of a linked allocation scheme, by working with *clusters*. A single cluster consists of a fixed number of blocks (e.g. four). This way, there is less capacity loss because of the pointer and the seek times are significantly smaller for sequential and direct reads. Clusters are supported in most operating systems.

File allocation table

The file allocation table (FAT) solution is actually a variant of the linked allocation. The difference is that now a separate table is kept which keeps a field per disk block. Each field contains the number of the next block that is part of the file this block belongs to. The number of the starting block is used as an index to this table. If the block is not used, it can be indicated by a 0 value. If we need to allocate a new block, we can simply locate the first field containing a 0 value. A better alternative is to link the free blocks to each other, like the blocks of the files are linked and to keep a pointer to the start of the free block list.

The advantage of the FAT approach is that this table is located on the initial part of the disk, which results in a relatively low seek time for direct access. This is because all the pointers to follow are contained in the FAT list, eliminating the possibility of them being scattered all over the disk. Furthermore, often part of the FAT will be stored in a cache to further reduce access times. For sequential reads, this is especially important because the disk head would otherwise be

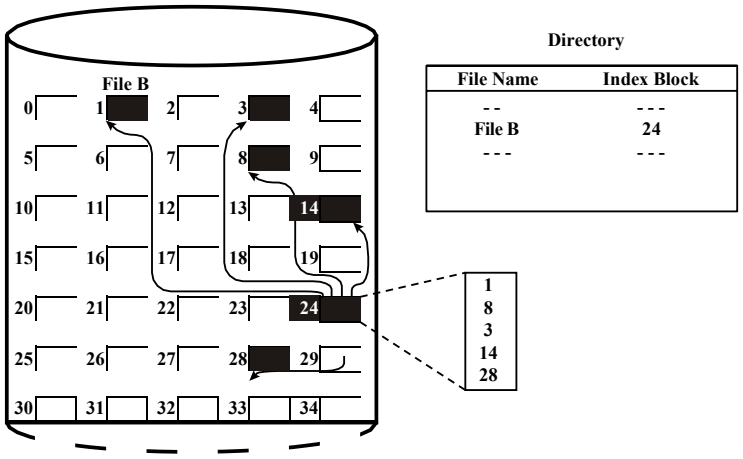


Fig. 59: Indexed file allocation [Stallings 98].

often have to move back and forth between the FAT and the actual disk blocks containing the data. MS-DOS and OS/2 both worked with file allocation tables.

Indexed allocation

Linked allocation solves most of the problems of continuous allocation. However, as indicated, direct access is particularly slow, because the seek times can be very high. The FAT structure can already greatly reduce the severity of this problem. An alternative solution is to provide an index block for each file. This block consists of a list of block numbers, where the *i-th* entry indicates the location of block *i* on the disk. In this case the directory contains the location of the index block (see Figure 59).

Indexed allocation clearly also allows very fast direct access since we only have to follow two pointers to locate the required disk block. The sequential read operations are still slower than a continuous allocation solution. Again, this can be partially solved by using clustering. Also note that indexed allocation is very similar to the paging mechanism that is typically used for memory management.

The main drawback of indexed allocation is the fact that each file has an index

block, while a file may consist of only one or two disk blocks. In this case, the index table contains only one or two entries that are different from zero. Therefore, clearly more capacity is used by small files than with a linked allocation.

This issue also raises the question of how many entries an index table should actually contain. In other words, should we provide more than one block to support large files? There are a number of possible solutions. First, if a single block is not enough, we could provide a linked list of index blocks. The disadvantage of this is that the number of pointers to follow in case of a reference to a block at the end of a very large file can quickly increase.

It is therefore preferable to work with a multilevel index (for large files), as was the case with the multilevel page table. This way, we can support very large files, while limiting the number of pointers to two or three. For example, if a pointer occupies 4 bytes and each block contains 4 sectors of 512 bytes each, a 2 level index alone can address a total of $512^2 = 262144$ blocks, which corresponds to about half a gigabyte. One can also use a combined scheme as we will see in a moment with the UNIX inode.

3.2 UNIX inodes

An interesting example of a graph-structured directory that uses an in- dexed allocation method can be found in UNIX systems. Here, an *inode will* be associated with each file (the structure of which can be found in Figure 60). The word i-node is an abbreviation for index node, where the hyphen is lost with time.

An inode contains all file attributes in its first fields. For instance, the *mode* displays information related to the protection of the file, the *owners* displays the ID of the owner and the group he or she belongs to, the *count* counts the number of hard links to the file, etc. The directory file is thus reduced to a pure translator of file names to inode numbers. The advantage of this approach is that creating hard links is very easy. One simply creates an entry in the directory file which also points to the corresponding inode and increases the *count* value of this node by one. A directory file is also an inode. Its content is equal to a list of inode numbers with their corresponding file names.

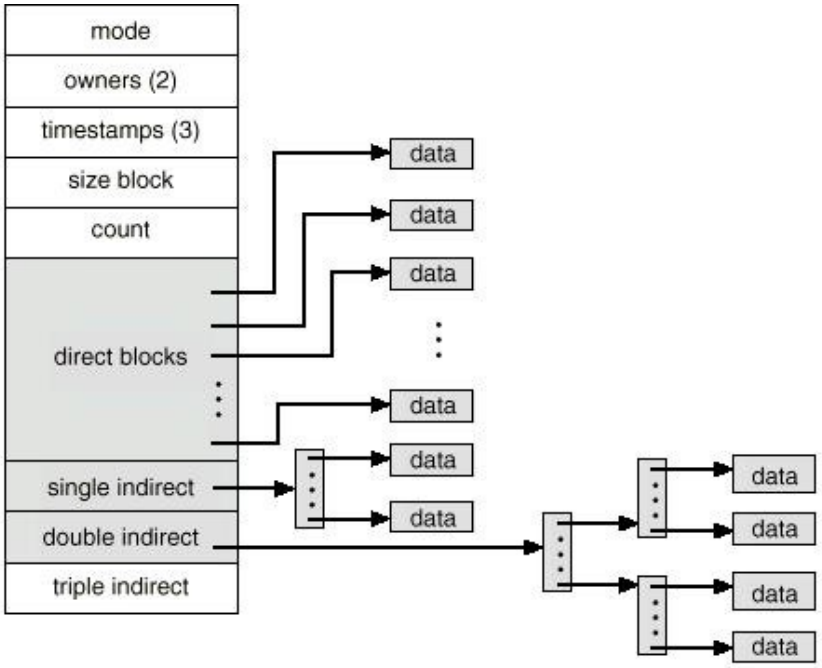


Fig. 60: The UNIX inode structure [Silberschatz 94].

As for the allocation method, we see that the inode uses indexed allocation. However, only for the first 12 (or 10) blocks there is a direct pointer from the index number to the associated disk block with the file data. The advantage of this is that the data blocks of small files are immediately available. However, providing direct pointers for all blocks would result in a very large inode (where it is unclear how many fields we should support).

That is why indirect pointers are used. After the 12 (or 10) direct pointers, there is one indirect pointer, which we call the *single indirect* pointer, pointing to a block consisting only of pointers to disk blocks. The next pointer in the inode, the *double indirect* pointer, points to an index block whose entries point to other index blocks. Finally, there is also a *triple indirect* pointer, which requires a total of four pointers to be followed before we can access the actual data of the file.

In this way, the amount of disk space a file needs is

indexing in a proper way related to the file size. For example, if there are 10 direct pointers, each pointer is 8 bytes in size and a block is 4Kbyte, we can address 40Kbyte directly, 2Mbyte via the single indirect pointer, 1Gbyte via a double indirect index and finally 0.5Tbyte via the triple indirect pointer.

The file systems *ext2* and *ext3* (popular in Linux) use this allocation method. Ext4 however works with *extents*. An extent is a cluster of consecutive blocks on disk and each inode contains a direct reference to 4 extents. When more than 4 extents are needed, the location of the remaining blocks is kept on disk (based on H-trees). Working with extents is much more efficient for large files that are almost not fragmented.

3.3 Free-space management

One problem we have not discussed so far is how to locate free space on disk when creating a new file (or when an existing file grows in size). Managing this free space is called *free-space management*.

Bit vector or bit map

A first regularly used method is to keep track of a binary vector that indicates which disk blocks are still free. Consequently, the length of the bit vector corresponds to the number of blocks n on the disk. If block number k *is* still free, then the k-*th* bit is equal to one.

This method is primarily driven by the bit manipulators available in the hardware of some computer systems (such as the Intel 80386 and 80486 and some PowerPC Macintosh systems). On some of these systems, one will read the bit map word by word into the processor and test each time if the word equals zero. If this is not the case, the position of the first 1 bit is located by means of an efficient hardware instruction. Finding the first free block can be done so quickly and it is also easy to locate a sequence of free blocks on the disk.

Due to the size of modern disks, the bit vector quickly becomes very large, which is disadvantageous because it should always be present in the memory. Clustering can be a solution again in this case,

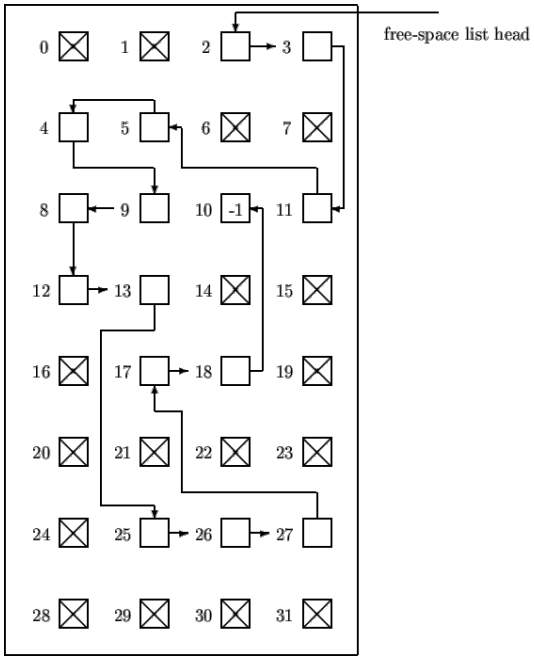


Fig. 61: Free space management via a linked list

where one will typically use larger clusters as the partition of the disk gets larger.

Linked list

An obvious alternative approach to managing free space is, as with linked allocation, to maintain a linked list of all free data blocks (see Figure 61).

In this case, the block number of the first free block is kept in a separate place on the disk, as well as in memory (for faster access). If a single free block is needed, we just have to follow this pointer. If a file is wiped, we have to add the blocks it took to the end of the list (which is certainly fast if the allocation method was also linked).

With this method, it is not easy to locate a sequence of consecutive free blocks, because the disk blocks do not have to be present in the list in ascending order. Also note that, for the FAT method, this linked list is an integral part of the file table.

Indexing

Finally, you can also use indexing to keep track of free disk space, by saying that free space is a file like any other file. In this case, it may be useful to shorten the index somewhat, providing only one index for each set of consecutive free blocks.

4 Fast file system for UNIX

We end this chapter by focusing on some of the key factors that will make a file system performant, meaning that data can be written to and from the disk at a high rate. We will do this using the Fast File System (FFS) for UNIX, presented in the ACM Transactions on Computer Systems journal in 1984.

4.1 The traditional UNIX file system

Like all UNIX file systems, the traditional UNIX file system developed by Bell Laboratories, uses inodes. It also maintained a linked list of free blocks. All blocks had an iden- tial size of 512 bytes (and somewhat later 1024 bytes). This implied that all data exchanges to and from the disk happened in blocks of 512 bytes.

This traditional approach turned out to be insufficient for many file system intensive applications (such as image processing). It turned out that the realised throughput was only 2% of the disk bandwidth (the bandwidth is equal to the number of bytes per track multiplied by the number of revolutions per second). The main explanation for this was the relatively small block size of 512 bytes, which was quickly increased to 1024 bytes to achieve a throughput of 4%. However, equally important was the use of the linked list for free-space ma- nagement. When setting up the file system, the successive free blocks in this free-space list were located close to each other on the hard disk. However, after a few weeks of average use, it appeared that these consecutive blocks had been transformed by the frequent deletion and creation of files.

to a list of blocks spread almost randomly across the disk. This created much larger seek times, reducing thoughput over this period by a factor of 6 (from 175 Kbytes/sec to 30 Kbytes/sec). Note that due to the resulting random nature of the blocks making up files, support for the read-ahead function also became useless.

Another disadvantage of the traditional file system was the way the inodes and data (within the partition covered by the file system) were organized. A traditional 150 Mbyte partition consisted of 4 Mbyte space for inodes, followed by 146 Mbyte for data. As with FAT, this leads to large seek times between the reading of the inode information and the requested data itself. Also, files belonging to the same directory often had widely spaced inodes, causing additional delays for applications that used multiple files in the same directory.

4.2 The Fast file system

Considering the disadvantages of the traditional approach, the solution seems obvious: we need to use a larger block size. This not only increases the throughput, but also allows to access larger files without using the *triple indirect* pointer in the inode. For example, blocks of 4096 bytes (and pointers of 4 bytes) give rise to $(1024)^2$blocks that we can address via a double indirect pointer, each block containing 4096 bytes, which results in supporting files up to 4 Gbyte. The problem with this approach is the large loss of storage capacity, because all files, including the many small files, are allocated a multiple of 4096 bytes. As can be seen in Figure 62, which was prepared from actual measurements on a multiuser system, this can result in a capacity loss of over 45% of the disk. The Fast file system for UNIX will solve this problem by also dividing each block into a number of fixed-size fragments (typically 2, 4 or 8, so that the fragments are still at least 512 bytes in size, each of which is addressable separately). However, before we go into that, let's discuss some general features of the new system.

%	Loss Organisation
0Single	Data, no space between traffic jams
4.2Single	Data, files start at 512 byte boundaries
	6.9Data + inodes, 512 byte block UNIX system
	11.8Data + inodes, 1024 byte block UNIX system
22.4Data	+ inodes, 2048 byte block UNIX system
45.6Data	+ inodes, 4096 byte block UNIX system

Fig. 62: Amount of space lost as a function of block size.

Data organisation

The FFS works with blocks of at least 4096 bytes, whereby the block size must be chosen when setting up the file system. Furthermore, the system will divide the disk partition to which the file system relates into a number of *cylinder groups*. Such a group consists of a series of adjacent cylinders of the hard disk (which may consist of a number of disks stacked on top of each other, hence the name cylinder). Within each cylinder group, one inode is provided per 2048 bytes (which should be more than enough) and a *bitmap is* kept indicating where the free space of this cylinder group is located.

However, this bitmap contains several bits per block, being the number of fragments into which a block is divided. Suppose the fragments are 1024 bytes in size, then we can ask ourselves how the operation of this system differs from a classical solution with blocks of 1024 bytes. To understand this, we need to take a closer look at the way these fragments are handled. Suppose, for example, that there are 4 fragments per block and that fragments 6 to 9 are still free, as well as fragments 12 to 15. The bitmap for the fragments 0 to 15 will then look like

1111 1100 0011 0000.

Although each set of four consecutive fragments corresponds to a single block in size, fragments 6 to 9 will not be considered a block since the number of the first fragment is not a multiple of 4.

Data allocation

The allocation proceeds as follows. Suppose we want to add data to a file, then the following three possibilities occur:

1. There is still enough space available for the extra data in the last fragment/block of the file. In this case, we simply add the data without allocating extra space.
2. If the space allocated to the file consists only of complete blocks, then we first fill the current block, then we add a number of complete blocks if necessary. Finally, we look for a block in which there are still enough free consecutive fragments to store the remaining part of the file (and which is possibly already partly used by another file).
3. The space already allocated to the file consists of a number of blocks plus a minimum of 1 and a maximum of 3 fragments. In this case we also consider these fragments as new data and use the procedure from possibility 2 to perform the allocation. This means that these fragments will be copied to another location.

The difference with a classic UNIX system with blocks of 1024 bytes is therefore mainly in the way we expand files if possibility 3 arises. How often we have to move fragments depends very much on the way the applications write or request data. Standard Input/Output library functions will always try to work with multiples of the used block size.

Free capacity reserve

Furthermore, the FFS will always keep a *reserve* of 5 to 10% of the available capacity to prevent the disk from becoming completely filled. The latter is necessary to avoid that free space requests would take too much time. The reserved percentage for free space can only be changed by the system administrator. Of course, this reserve must be counted as loss capacity of the file system. From Figure 62 we can see that a system with blocks of 4096 bytes and fragments of 512 bytes, results in a loss of about 6.9%. So we can easily add another 5% to the reserve

without sacrificing more capacity than in a traditional 1024-byte block system (with a loss of 11.8%).

Block and inode selection

By using blocks of 4096 bytes, the reading and writing speed will increase considerably. However, this alone is not enough to achieve a very good performance. One must also ensure that the seek times between the reading/writing of successive blocks on the disk are limited. Without dwelling on the details, the FFS will try to place the first 48 Kbyte (that is, the data contained in the 12 direct blocks of the disk) in the same cylinder group. After that, a new cylinder group will be created for each additional Mbyte. This way, it is avoided that one single file takes up too much space in a cylinder group, preventing the other files in this group from growing. We also try to continue with a group of cylinders that have a lower occupation rate than average. In short, we should try to avoid that groups of cylinders are filled up completely, because this will lead to more seek times when we want to enlarge a file in the group.

As far as the assignment of inodes to newly created files is concerned, the FFS will try to place the inodes of files belonging to the same directory within a single cylinder group. In contrast, inodes for directory files are sought in cylinder groups for which the number of free inodes is even greater than the average.

Performance

To evaluate the effectiveness of the FFS, we will compare its performance with that of the traditional UNIX system. Note that the results refer to a 1983 system that had been in use for over a month before the measurements were taken. The results for read and write speeds are shown in Figure 63. The only difference between the UNIBUS and MASSBUS tests is the hard disk controller used. In the past the disk controller was not an integral part of the hard disk, but was often located on a separate controller card. This allowed a single disk to be combined with several controllers.

% File system	Bus	Reading rate	Reading bandwidth	% CPU
old 1024	UNIBUS	19 Kb/sec	29/983 (3%)	11%
FFS 4096/1024	UNIBUS	221 Kb/sec	221/983 (22%)	43%
FFS 8192/1024	UNIBUS	233 Kb/sec	233/983 (24%)	29%
FFS 4096/1024	MASSBUS	466 Kb/sec	466/983 (47%)	73%
FFS 8192/1024	MASSBUS	466 Kb/sec	466/983 (47%)	54%
% File system	Bus	Write speed	Writing band width	% CPU
old 1024	UNIBUS	48 Kb/sec	48/983 (5%)	29%
FFS 4096/1024	UNIBUS	142 Kb/sec	142/983 (14%)	43%
FFS 8192/1024	UNIBUS	215 Kb/sec	215/983 (22%)	46%
FFS 4096/1024	MASSBUS	323 Kb/sec	323/983 (33%)	94%
FFS 8192/1024	MASSBUS	466 Kb/sec	466/983 (47%)	95%

Fig. 63: Read and write speed, realized bandwidth and CPU load on UNIX systems (VAX 11/750)

We see that the FFS is capable of much higher read and write speeds, with throughputs up to 47%. In contrast to the old system, the throughput has not decreased compared to shortly after the creation of the file system. When we take a closer look at the results from Figure 63, we see a number of remarkable things.

The reading speed in the traditional system is lower than the writing speed, which is remarkable since the CPU has (twice) more work when writing data than when reading. However, this is explained by the fact that the read commands on this system are *synchronous*, which means that the read commands are processed in sequence. Writing however can be *asynchronous*, so the CPU, which has spare capacity, can generate write requests faster than data can be written to the disk. As a result, multiple writes are stored in the disk cache, which results in a more optimal order (i.e., with lower seek times). In the FFS system, the situation is completely different since the CPU has become the slowing down factor instead of the disk. Furthermore, the write commands are also presented in a much better order (thanks to the introduction of cylinder groups), so that the possibility of handling them asynchronously is of little additional benefit. Consequently, read commands are again faster, as one would expect. Note, that physically writing or reading data on disk is

of course equally fast and is determined by the rotational speed of the disk and the amount of data per track.

5 Journaling file systems

Today's file systems use *journaling* with the goal of quickly recovering the file system after a crash or power outage. To see the need for a journal we use the popu- larly Linux ext2 file system (that is, the second Extended File System). As in the FFS, the disk blocks in ext2 are divided into several groups, each group consisting of a number of inodes and data blocks. Furthermore, each group also contains a bitmap indicating which inodes within the group are in use and a bitmap indicating the same for the data blocks.

Data inconsistency and nonsensical data

Now when we look at a file that consists of a single data block (meaning only the first direct pointer in the inode points to a data block) and we want to add a data block to this file, three changes have to be made on disk. First the data must be written in a free data block, next the second direct pointer in the inode of the file must point to this data block and finally the bitmap indicating which data blocks are in use must be modified. So three writes must be performed and the controller of the disk will usually determine the order of these writes.

Now suppose that a crash or power failure occurs before all three writes have taken place, then the following cases may occur:

1. Suppose only the bitmap or the inode has been modified. In the first case the bitmap indicates that the data block is in use, but there is no inode that refers to it. In the second case, there is a referring inode, but the bitmap indicates that the data block is still free. In both cases we see that the file system is no longer consistent.

2. If only the data block is written, then the file system is still consistent, but the written data is lost (since there is no

inode refers to the data and we also don't know which data block it refers to).

3. If both the bitmap and the inode are modified, without writing the data, the system is again consistent. However, another problem arises: the content of the data block to which the inode refers is not modified and thus contains garbage.

4. If only the bitmap or inode is not changed, there is again an inconsistency in the file system.

In older file systems (like ext2), possible inconsistencies in the file system were often solved at reboot time by using specific tools (e.g., fsck in ext2). For example, if a certain data block is not free according to the bitmap, but no inode refers to this block, we can easily solve this by adjusting the bit in the bitmap. However, it is clear that detecting and solving inconsistencies in the file system is a slow operation given the size of today's hard disks. Hence, more recent file systems avoid the need for such tools by using a *journal*. This is also the main difference between the Linux ext2 and ext3 file system.

Possible solution

The idea of journaling originated in database management and involves specifying on disk what changes we want to make before we actually make them. In other words, before we make changes on disk, we write what we're going to do elsewhere on disk. The place where we write the changes is called the *log* or *journal*. That's why journaling is called *write-ahead logging*.

So in our example, we will first specify the three required writes as a single *transaction* in the log. Once this transaction is added to the log, we execute the transaction and then remove the transaction from the log. If a crash or power outage occurs, it is sufficient to re-execute the transactions in the log (that is, a replay of these transactions occurs) to make the system consistent.

Some caution is required when writing the transactions to the log. Writing a transaction also corresponds to

multiple writes, where there is a write that writes a control block at the beginning of the transaction and a write that writes a block marking the end of the transaction. It is important that this last write happens last because once the control block marking the end of the transaction is added to the log, the transaction is considered valid (that is, the transaction is committed). However, if all remaining writes of the transaction have not yet been performed and the system fails, the replay of the log structure may result in an inconsistency or in nonsensical data in the file. This means that the write indicating the end of the transaction will not be forwarded until all remaining writes of the transaction have been performed. We also note that writing a single block to a disk may be considered an atomic operation.

While keeping a journal allows us to quickly make the data consistent, it also has a cost: we have to do all the writes twice, which is very expensive. In practice, there are several modes of journaling (in ext3, there are 3 modes) and the one we have described so far is known as *data journaling*. A less expensive mode of journaling, called *metadata journaling*, consists of writing only the *metadata* in the log, and writing to the data blocks immediately at their final location. Although metadata journaling more often results in nonsense data, this mode does not require that all data be written twice. In our example the difference seems small since we only had one data block, but in practice the file system will group multiple writes to a single transaction making the gain much greater.

Finally, there is a distinction between *ordered* and *unordered* metadata journaling. In ordered metadata journaling, all data blocks are written to their final location. Only then is the transaction added to the log. Next, the metadata is written definitively and the transaction is removed. In the unordered metadata journaling mode, the transaction is first written to the log, then the transaction is executed and the data blocks are modified at the same time, and finally the transaction is deleted from the log. Clearly, the unordered mode will result in more nonsensical data than the ordered mode.